

ArthBodh: Context-Aware Meaning Finder for Hindi Words

Oct 8, 2026 · @Sandip

1. Project overview

ArthBodh (अर्थबोध, "understanding of meaning") lets a user enter any sentence, in English or Hindi, pick any Hindi word in it, and see the one correct meaning of that word in that sentence.

Full title: ArthBodh: Context-Aware Meaning Finder for Hindi Words using Knowledge-Based Word Sense Disambiguation

The problem

Many Hindi words have several unrelated meanings. A dictionary lists all of them and leaves the reader to guess; translators often guess wrong on short sentences. ArthBodh picks the right meaning automatically from context, for any word and any sentence.

Who it helps: Hindi learners, students from non-Hindi states, people reading Hindi notices and forms, and translators checking ambiguous words.

Two input modes

Mode	User types	What happens
English mode	Any English sentence	Translated to Hindi; the user picks a Hindi word; both the Hindi and English sentences are used as context
Hindi mode	Any Hindi sentence	The user picks a word; only the Hindi sentence is used as context (the pure WSD test)

What the user sees

Example (English mode): "The farmers got a good result for their hard work."

1. Hindi translation appears with every word as a clickable button, e.g. किसानों को उनकी मेहनत का अच्छा फल मिला।

2. The user clicks फल.

3. ArthBodh shows:

- **Meaning in this sentence:** परिणाम / result, outcome
- **Other meanings of फल:** fruit (of a plant), blade of a weapon, etc.
- **Confidence:** how sure the system is
- **Compare methods:** what Simplified Lesk and Embedding Lesk each chose

What we deliver

- A WSD engine that works for **any Hindi word in IndoWordNet**, using two methods we implement: Simplified Lesk and Embedding Lesk
- English-to-Hindi translation and a word picker in a Streamlit web app
- An automatically built evaluation set and a hand-labelled Hindi test set, with accuracy results for both methods
- A project report and presentation

2. Key concepts in simple words

Read this section first: these ideas are the whole project.

Word Sense Disambiguation (WSD)

A **sense** is one meaning of a word. **WSD** means choosing the correct sense of a word from the words around it (its **context**). In "सोने का हार", हार sits next to सोना (gold), so it means necklace, not defeat.

Why "any word" needs a knowledge-based approach

A trained (supervised) model only knows the words it was trained on. Since the user can pick any word in any sentence, ArthBodh uses a **dictionary as its knowledge**: IndoWordNet. No training per word is needed.

IndoWordNet

A lexical database of Indian languages built at IIT Bombay, structured like English WordNet. For each Hindi word it lists its senses (**synsets**); each sense has a **part of speech**, a **gloss** (definition) and **example sentences**. We use it through the `pyiwn` Python library.

Lemma and part of speech (POS)

IndoWordNet stores base forms, so किसानों must become किसान and सोने must become सोना before lookup. This is **lemmatization**. We also tag the word's **POS** (noun, verb, adjective): सोना as a noun can mean gold, as a verb it means to sleep. Keeping only senses with the matching POS removes wrong options before disambiguation even starts. We use the Stanza library for both.

Method 1: Simplified Lesk (our baseline)

The correct sense is the one whose gloss and examples share the most words with the sentence. Simple and explainable, but fails when the sentence and gloss use different words for the same idea.

Method 2: Embedding Lesk (our main method)

Same idea, but instead of counting identical words we convert the sentence and each gloss into **vectors** (lists of numbers that capture meaning) with LaBSE, a pretrained multilingual model. The sense whose gloss vector is closest to the sentence vector (**cosine similarity**) wins. It understands that मीठा and पका relate to fruit even without exact overlap.

Cross-lingual context (English mode only)

LaBSE places English and Hindi text in the same vector space. So in English mode we compare each Hindi gloss with both the Hindi translation and the original English sentence, and combine the scores. The English sentence is an extra, clean clue, especially useful when the translation is imperfect.

Translation

English to Hindi translation is the input step, not our NLP contribution. We use a ready translator (Google Translate through `deep-translator`, or AI4Bharat's IndicTrans2 for an offline option).

3. Syllabus mapping

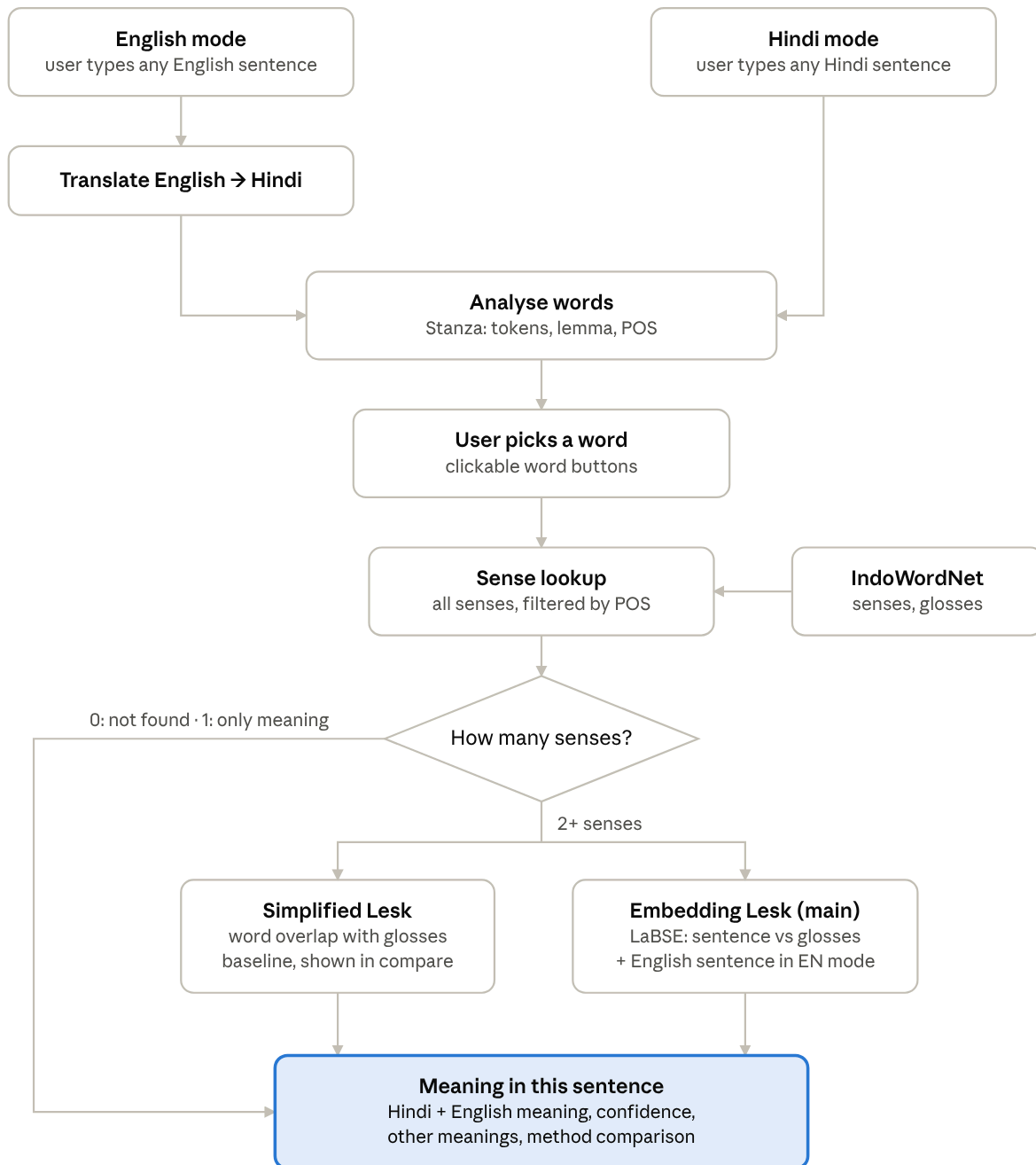
ArthBodh uses topics from all six modules.

Module	Syllabus topic	Where we use it
1. Introduction	Ambiguity in natural language, levels of NLP	The core problem: lexical ambiguity
2. Word Level Analysis	Tokenization, lemmatization, morphological analysis	Splitting the sentence into words; reducing किसानों → किसान before dictionary lookup
3. Syntax Analysis	Part-of-speech tagging	Tagging the chosen word's POS and filtering senses by it
4. Semantic Analysis	Lexical semantics, homonymy, polysemy, WordNet	IndoWordNet senses, glosses and relations
4.2	WSD, knowledge-based approach (Lesk's algorithm)	Simplified Lesk (baseline) and Embedding Lesk (main method)
5. Pragmatics & Discourse	Context and coherence	Using the whole sentence, and in English mode the original sentence, as context
6. Applications	Machine translation, Indian regional languages	English → Hindi translation; the whole system works on Hindi

4. End-to-end architecture

Both input modes end up as a Hindi sentence; from there one pipeline analyses the words, looks up the chosen word's senses, and picks the meaning that fits.

Any sentence, any word: two input modes meet in one disambiguation pipeline



ArthBodh architecture · English and Hindi modes, one pipeline

Nothing is trained per word: every decision is computed at runtime from the sentence and IndoWordNet, which is why it works for any word.

Components

Component	What it does	File
Input	Mode switch (English / Hindi) and a text box	<code>app.py</code>
Translator	English → Hindi sentence; Hindi gloss → English for display	<code>translate.py</code>
Word analyser	Splits the Hindi sentence into words with lemma and POS	<code>nlp.py</code>
Word picker	Shows each Hindi word as a button; the user clicks one	<code>app.py</code>
Sense lookup	Fetches the word's senses from IndoWordNet and keeps those matching its POS	<code>wordnet.py</code>
Simplified Lesk	Word-overlap baseline, shown in compare mode	<code>lesk.py</code>
Embedding Lesk	Main method; compares sentence (and English sentence in English mode) with each gloss	<code>embed_lesk.py</code>
Engine	Runs the steps for one sentence + word and packages the result	<code>engine.py</code>
Output	Meaning in Hindi and English, confidence, other meanings, method comparison	<code>app.py</code>

Walk-through: English mode

Input: **"The farmers got a good result for their hard work."**

1. Translate → किसानों को उनकी मेहनत का अच्छा फल मिला।
2. Analyse words → फल: lemma फल, POS NOUN
3. User clicks फल → IndoWordNet returns its noun senses (fruit, result, blade, ...)
4. Embedding Lesk compares the glosses with the Hindi sentence and the English sentence → *result / outcome* scores highest
5. Output: फल = परिणाम (result), with confidence and the other meanings

Walk-through: Hindi mode

Input: **"बाज़ार में ताज़े फल बिक रहे हैं।"**

No translation step. The user clicks फल; Embedding Lesk uses only the Hindi sentence; words like बाज़ार and ताज़े pull it to *fruit*.

5. Resources and test data

ArthBodh needs no training data at all; data is only needed to measure how accurate it is.

Resources the system uses

Resource	What it gives us
IndoWordNet (via <code>pyiwn</code>)	Senses, POS, glosses and examples for tens of thousands of Hindi words
LaBSE (via <code>sentence-transformers</code>)	Pretrained multilingual sentence vectors for English and Hindi
Stanza Hindi model	Tokenization, lemmas and POS tags
Translator (<code>deep-translator</code> or <code>IndicTrans2</code>)	English → Hindi sentences, and Hindi glosses → English for display
Hindi stop word list	Words like है, का, को, ने removed before Simplified Lesk

Test set 1: auto-built from a parallel corpus (large)

The IIT Bombay English–Hindi Parallel Corpus has about 1.49 million Hindi sentences with English translations. We use the English side as a free answer key.

1. Choose about 30 common ambiguous Hindi words (आम, हार, फल, सोना, उत्तर, बाल, पत्र, तीर, कल, कर, मूल, ...).
2. For each word, map each IndoWordNet sense to English keywords once, by hand.
Example for फल: fruit sense → "fruit"; result sense → "result", "outcome".
3. Pick sentence pairs containing the word; label each by which keyword appears in the English side; drop unclear ones.
4. Keep about 50 sentences per word (roughly 1,500 total).
5. Hand-check 100 random items and report how accurate the auto-labels are.

Test set 2: hand-labelled Hindi sentences (realistic)

150 to 200 Hindi sentences from Hindi news sites and Hindi Wikipedia, containing ambiguous words of our choosing (not limited to the 30). Two members label the correct IndoWordNet sense independently, then resolve disagreements.

File format

```
hindi_sentence,english_sentence,target_word,gold_sense_id,source
किसानों को मेहनत का अच्छा फल मिला।,The farmers got a good result for their hard
work.,फल,<synset id>,auto
बाज़ार में ताज़े फल बिक रहे हैं।,,फल,<synset id>,manual
```

The English column is empty for hand-labelled Hindi sentences, which are tested in Hindi mode only.

6. Tools, tech stack and folder structure

Everything is free and runs on a laptop without a GPU; Google Colab works for heavy steps.

Tool	Purpose	Install
Python 3.10+	Main language	python.org
pyiwn	IndoWordNet senses, POS, glosses, examples	<code>pip install pyiwn</code>
stanza	Hindi tokenization, lemmas, POS tags	<code>pip install stanza</code>
sentence-transformers	LaBSE embeddings for Embedding Lesk	<code>pip install sentence-transformers</code>
deep-translator	English → Hindi and Hindi → English translation	<code>pip install deep-translator</code>
pandas, scikit-learn	Test data handling and metrics	<code>pip install pandas scikit-learn</code>
matplotlib	Result charts for the report	<code>pip install matplotlib</code>
Streamlit	ArthBodh web app	<code>pip install streamlit</code>

Tool	Purpose	Install
GitHub	Shared code repository	github.com

One-time downloads: the LaBSE model (around 2 GB) and the Stanza Hindi model; do both on good Wi-Fi. `deep-translator` needs internet during the demo. Check the pyiwn docs at the start, since function names in the code below may differ slightly by version.

Folder structure

```

arthbodh/
├── data/
│   ├── raw/                # parallel corpus files
│   ├── sense_keywords.json # sense → English keywords for auto test set
│   ├── test_auto.csv       # test set 1
│   ├── test_manual.csv     # test set 2
│   └── stopwords_hi.txt
├── src/
│   ├── translate.py        # English → Hindi, gloss → English
│   ├── nlp.py              # tokenize, lemma, POS (Stanza)
│   ├── wordnet.py          # IndoWordNet lookup, POS filter, cache
│   ├── lesk.py             # Simplified Lesk
│   ├── embed_lesk.py       # Embedding Lesk (Hindi + cross-lingual)
│   └── engine.py           # ties everything together: analyse(sentence,
word)
├── build_testset.py        # builds test set 1 from the corpus
├── evaluate.py             # accuracy and charts
├── app.py                  # Streamlit app
└── requirements.txt

```

7. Step-by-step implementation

Build the modules in this order and test each on its own before moving on.

Step 1: Translation (`translate.py`)

```
from deep_translator import GoogleTranslator

def en_to_hi(text):
    return GoogleTranslator(source='en', target='hi').translate(text)

def hi_to_en(text):          # used to show glosses in English
    return GoogleTranslator(source='hi', target='en').translate(text)
```

Step 2: Tokens, lemmas and POS (`nlp.py`)

```
import stanza
# run once: stanza.download('hi')
nlp = stanza.Pipeline('hi', processors='tokenize,pos,lemma')

def analyse_words(sentence):
    doc = nlp(sentence)
    return [{'text': w.text, 'lemma': w.lemma, 'upos': w.upos}
            for s in doc.sentences for w in s.words if w.upos != 'PUNCT']
```

Each word now has its surface form (किसानों), lemma (किसान) and POS (NOUN).

Step 3: IndoWordNet lookup with POS filter (`wordnet.py`)

```
import pyiwn
iwn = pyiwn.IndoWordNet()          # Hindi by default
POS_MAP = {'NOUN': 'noun', 'PROPN': 'noun', 'VERB': 'verb',
            'ADJ': 'adjective', 'ADV': 'adverb'}
CACHE = {}

def get_senses(word, upos=None):
    if word not in CACHE:
        try:
            CACHE[word] = [{'id': s.synset_id(), 'pos': str(s.pos()).lower(),
                            'gloss': s.gloss(), 'examples': s.examples(),
                            'lemmas': s.lemma_names()} for s in
            iwn.synsets(word)]
        except Exception:
            CACHE[word] = []
    senses = CACHE[word]
    wanted = POS_MAP.get(upos)
    same_pos = [s for s in senses if wanted and wanted in s['pos']]
    return same_pos or senses      # fall back to all senses if POS filter
    removes everything
```

Look up the lemma first; if it returns nothing, try the surface form. Print `s.pos()` for a few words first to confirm pyiwn's POS labels and adjust `POS_MAP`.

Step 4: Simplified Lesk (`lesk.py`)

Written by us, no ready-made Lesk function.

```

STOP = set(open('data/stopwords_hi.txt', encoding='utf-8').read().split())

def words_of(text):
    return {w['lemma'] for w in analyse_words(text) if w['lemma'] not in
STOP}

def simplified_lesk(sentence, target_lemma, senses):
    context = words_of(sentence) - {target_lemma}
    overlaps = [len(context & words_of(s['gloss'] + ' ' + '
'.join(s['examples'])))
                for s in senses]
    best = max(range(len(senses)), key=lambda i: overlaps[i])
    return best, overlaps

```

Comparing lemmas instead of raw words means किसानों in a sentence still matches किसान in a gloss.

Step 5: Embedding Lesk with cross-lingual context (embed_lesk.py)

```

from sentence_transformers import SentenceTransformer, util
model = SentenceTransformer('sentence-transformers/LaBSE')

def embedding_lesk(hi_sentence, senses, en_sentence=None, w_en=0.5):
    gloss_texts = [s['gloss'] + ' | ' + ' '.join(s['examples']) for s in
senses]
    G = model.encode(gloss_texts, convert_to_tensor=True)
    scores = util.cos_sim(model.encode(hi_sentence, convert_to_tensor=True),
G)[0]
    if en_sentence:
        # English mode
        en_scores = util.cos_sim(model.encode(en_sentence,
convert_to_tensor=True), G)[0]
        scores = (1 - w_en) * scores + w_en * en_scores
    best = int(scores.argmax())
    return best, scores.tolist()

```

Confidence = the gap between the best and second-best score. A big gap means a clear decision; a small gap means the system is unsure, and the app says so. Tune `w_en` (0.3, 0.5, 0.7) on test set 1.

Step 6: The engine (engine.py)

```
def analyse(hi_sentence, word, en_sentence=None):
    senses = get_senses(word['lemma'], word['upos']) or
    get_senses(word['text'])
    if not senses:
        return {'status': 'not_found'}
    if len(senses) == 1:
        return {'status': 'single', 'sense': senses[0]}

    e_idx, e_scores = embedding_lesk(hi_sentence, senses, en_sentence)
    l_idx, overlaps = simplified_lesk(hi_sentence, word['lemma'], senses)
    top2 = sorted(e_scores, reverse=True)[:2]
    return {'status': 'ambiguous',
            'sense': senses[e_idx],
            'meaning_en': hi_to_en(senses[e_idx]['gloss']),
            'confidence': top2[0] - top2[1],
            'other_senses': [s for i, s in enumerate(senses) if i != e_idx],
            'lesk_choice': senses[l_idx], 'lesk_overlaps': overlaps}
```

Step 7: The Streamlit app (app.py)

```
import streamlit as st

mode = st.radio('Input language', ['English', 'Hindi'], horizontal=True)
text = st.text_input('Type a sentence')
if text:
    en = text if mode == 'English' else None
    hi = en_to_hi(text) if mode == 'English' else text
    st.subheader(hi)
    words = analyse_words(hi)
    pick = st.radio('Pick a word', range(len(words)),
                    format_func=lambda i: words[i]['text'], horizontal=True)
    result = analyse(hi, words[pick], en)
    # show: meaning (Hindi + English), confidence, other meanings,
    # and a 'Compare methods' expander with Simplified Lesk's choice
```

Load Stanza, LaBSE and IndoWordNet once with `@st.cache_resource` so the app stays fast after the first run.

Step 8: Build test set 1 (`build_testset.py`)

For each chosen word: read sentence pairs from the parallel corpus, keep pairs where the Hindi side contains the word and the English side contains keywords of exactly one sense (from `sense_keywords.json`), and save up to 50 per word.

Step 9: Evaluate (`evaluate.py`)

Run every method on both test sets and produce the results table and charts (section 8).

Step 10: Report and presentation

Problem, architecture diagram, methods, results, error analysis and app screenshots.

8. Evaluation plan

We compare a baseline and three system variants on both test sets, and report accuracy numbers for each.

What we compare

Variant	Context used
Baseline: first listed sense	None (always picks IndoWordNet's first sense for the POS)
Simplified Lesk	Hindi sentence, word overlap
Embedding Lesk, Hindi mode	Hindi sentence, LaBSE vectors
Embedding Lesk, English mode	Hindi + English sentences, LaBSE vectors

Metrics

- **Accuracy:** chosen sense = gold sense, overall and per word
- **Accuracy with vs without the POS filter**, to show what POS tagging adds
- **Confidence check:** accuracy on high-confidence vs low-confidence answers (a useful confidence score should separate them)
- **Published reference:** the HindiWSD paper (WILDRE 2022) reports about 34% for original Lesk and 71% for their customized Lesk on Hindi

Results table for the report

Variant	Test set 1: auto-built (%)	Test set 2: hand-labelled Hindi (%)
Baseline: first sense		
Simplified Lesk		
Embedding Lesk, Hindi mode		
Embedding Lesk, English mode		not applicable

English mode on test set 1 will look strong because the English side contains the answer keyword; report it, but treat Hindi mode and test set 2 as the honest measure.

Error analysis (important for the viva)

Pick 10 wrong answers and explain each. Expected causes:

- IndoWordNet lists very fine-grained senses that even humans confuse.
- Some glosses are short, giving Lesk little to match.
- Translation errors in English mode change the Hindi word or its context.
- Wrong lemma or POS from Stanza sends the lookup to the wrong senses.
- कल (yesterday vs tomorrow) depends on verb tense, which none of the methods models directly.

9. Demo plan and likely viva questions

Keep the demo under 5 minutes, and let the teacher type her own sentences.

Demo flow

1. One-line problem: "फल can mean fruit or result; which one is it here?"
2. **English mode:** type "The farmers got a good result for their hard work." Show the Hindi translation, click फल, show *result* with its confidence and other meanings.
3. **Hindi mode:** type "बाज़ार में ताज़े फल बिक रहे हैं", click फल, show *fruit*. Same word, different meaning, no English help.
4. Open "Compare methods" on a sentence where Simplified Lesk is wrong and Embedding Lesk is right; explain why.
5. Hand over the keyboard: the teacher types any sentence and picks any word.

6. Show the results table, one limitation and one future improvement.

Test 10 demo sentences the day before, and keep screenshots ready in case the internet fails.

Likely viva questions

Question	Short answer
What is WSD?	Choosing the correct meaning of an ambiguous word from its context.
Why knowledge-based, not machine learning?	The user can pick any word; a trained model only knows words it has training data for, while IndoWordNet covers tens of thousands.
How does Simplified Lesk work?	Picks the sense whose gloss shares the most words with the sentence.
How is Embedding Lesk better?	It compares meaning through vectors, so related words match even when they are not identical.
Isn't English mode cheating, since English already shows the meaning?	Translation hides the meaning again in Hindi; we use English only as extra context, and Hindi mode is evaluated separately as the pure test.
Why lemmatize and POS-tag?	IndoWordNet stores base forms, and POS removes wrong senses (सोना noun vs verb).
What does the confidence mean?	The gap between the best and second-best sense score; a small gap means unsure.
How did you evaluate without a big labelled dataset?	Built one automatically from a parallel corpus using English translations as the answer key, plus a hand-labelled Hindi set.
Is it hardcoded?	No. Every decision is computed from the sentence and the dictionary at runtime; the teacher can test any sentence.

Question	Short answer
Limitations?	Fine-grained dictionary senses, short glosses, translation errors, words missing from IndoWordNet.

10. Scope, risks and extensions

In scope / out of scope

- **In scope:** any English or Hindi sentence; any Hindi word that exists in IndoWordNet; two WSD methods; Streamlit app; evaluation on two test sets
- **Out of scope:** building our own translator, training a deep learning WSD model, languages other than Hindi

Risks and fallbacks

Risk	Fallback
pyiwn fails to install or its API differs	Pin the version that works; read the IndoWordNet data files directly if needed
Word not found in IndoWordNet	Try the surface form, then a simple suffix-stripped form; else show "meaning not available"
Translator needs internet and it fails in the demo	Switch to Hindi mode, or use IndicTrans2 offline; keep screenshots ready
LaBSE is slow on our laptops	Cache gloss vectors per word; run evaluation in Google Colab
Stanza gives a wrong lemma or POS	Fall back to all senses when the POS filter returns nothing; list it in error analysis
Accuracy is low on fine-grained senses	Report it honestly, and also report "correct within top 2" accuracy

Extensions if time allows

- Marathi support (IndoWordNet covers Marathi; same pipeline)

- Extended Lesk: add glosses of related synsets (hypernyms, hyponyms) to each sense
- Fine-tuned model comparison (e.g. MuRIL or IndicBERT) for a few words
- Browser extension: select a word on any Hindi web page to see its meaning in context

References

- Lesk, M. (1986). Automatic sense disambiguation using machine readable dictionaries.
- Jurafsky & Martin, *Speech and Language Processing*, chapter on Word Senses and WordNet.
- [HindiWSD: A package for word sense disambiguation in Hinglish & Hindi](#) (Yusuf et al., WILDRE 2022)
- [Context based Analysis of Lexical Semantics for Hindi Language](#) (Ansari & Khan, 2019)
- [The IIT Bombay English-Hindi Parallel Corpus](#) (Kunchukuttan et al., LREC 2018)
- IndoWordNet, Center for Indian Language Technology (CFILT), IIT Bombay; `pyiwn` Python interface
- Feng et al. (2022). Language-agnostic BERT Sentence Embedding (LaBSE).
- Stanza: A Python NLP toolkit for many human languages (Qi et al., 2020).